

Lab 13 - Sigma Detection Engineering

Windows PowerShell Detection Rule Development and Splunk Validation

Objective: Create a Sigma rule for suspicious encoded/Base64-related PowerShell activity, validate the rule structure, convert it to Splunk SPL using Sigma CLI, and test the detection against Windows PowerShell Event ID 4104 telemetry in Splunk.

Environment

Component	Purpose
Windows 11 VM	Rule authoring, PowerShell telemetry generation, Sigma CLI installation
PowerShell Script Block Logging	Provides Event ID 4104 script-block telemetry
Sigma CLI 3.1.0	Validates and converts Sigma rules
Splunk backend plugin	Converts Sigma detection logic into Splunk SPL
Splunk Enterprise	Searches and validates matching PowerShell telemetry

Sigma Rule Design

Detection goal: Identify PowerShell script-block content related to encoded commands and Base64 encoding/decoding.

title: Suspicious PowerShell Encoded Command Activity

logsource:

product: windows

category: ps_script

detection:

selection:

ScriptBlockText | contains:

- FromBase64String
- EncodedCommand
- ToBase64String

condition: selection

level: medium

tags:

- attack.execution
- attack.t1059.001

Field	Value
Rule status	Experimental
Log source	Windows PowerShell Script Block Logging
Detection field	ScriptBlockText
Match terms	FromBase64String, EncodedCommand, ToBase64String
Severity	Medium
ATT&CK mapping	T1059.001 - Command and Scripting Interpreter: PowerShell
Expected false positives	Legitimate administration, security tooling, authorized lab activity

```

- ToBase64String
condition: selection

falsepositives:
- Legitimate administrative scripts
- Security testing or lab activity

level: medium

tags:
- attack.execution
- attack.t1059.001

PS C:\WINDOWS\system32> & "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" convert -t splunk -
p splunk_windows C:\SOC-Lab\lab13_encoded_powershell.yml
Parsing Sigma rules [#####] 100%
ScriptBlockText IN ("*FromBase64String*", "*EncodedCommand*", "*ToBase64String*")
PS C:\WINDOWS\system32>

```

Figure 1 - Sigma rule stored as lab13_encoded_powershell.yml and successfully parsed by Sigma CLI.

Sigma CLI and Splunk Backend

Sigma CLI was installed with Python, and the Splunk backend plugin was added successfully. The executable was called directly because the Python Scripts directory was not yet in Windows PATH.

```
& "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" plugin install splunk
```

```

Administrator: Windows PowerShell

ppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
WARNING: The script normalizer.exe is installed in 'C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts' wh
ich is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
----- 20/21 [sigma-cli] WARNING: The script sigma.exe is installed in 'C:\Users\D
ulan\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed MarkupSafe-3.0.3 certifi-2026.7.22 charset_normalizer-3.5.2 click-8.5.0 colorama-0.4.6 diskcache-
5.6.3 diskcache-stubs-5.6.3.6.20240818 idna-3.20 Jinja2-3.1.6 jq-1.12.0 packaging-26.3 prettytable-3.18.0 pyparsing-3.3.
3 pysigma-1.5.1 pyyaml-6.0.3 requests-2.34.2 sigma-cli-3.1.0 types-pyyaml-6.0.12.20260906 typing-extensions-4.16.0 urlli
b3-2.8.0 wcwidth-0.9.1
PS C:\WINDOWS\system32> sigma plugin install splunk
sigma : The term 'sigma' is not recognized as the name of a cmdlet, function, script file, or operable program. Check
the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ sigma plugin install splunk
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (sigma:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\WINDOWS\system32> & "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" version
3.1.0 (online pypi.org: +[32m+[1m3.1.0+[0m)
PS C:\WINDOWS\system32> & "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" plugin install splu
nk
Successfully installed plugin 'splunk'
+ [32mpySigma version is compatible with sigma-cli+[0m
PS C:\WINDOWS\system32> & "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" version
3.1.0 (online pypi.org: +[32m+[1m3.1.0+[0m)
PS C:\WINDOWS\system32>

```

Figure 2 - Sigma CLI 3.1.0 and the Splunk plugin installed successfully.

Processing Pipeline Selection

The Splunk backend required a processing pipeline. Available pipelines were enumerated and **splunk_windows** was selected for standard Windows log-source mapping.

Pipeline	Purpose
splunk_windows	Standard Splunk Windows log source conditions - selected
splunk_sysmon_acceleration	Sysmon search acceleration keywords
splunk_cim	Splunk CIM data-model mapping

```

PS C:\WINDOWS\system32> & "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" list pipelines splunk
+-----+-----+-----+-----+-----+
| Identifier | Priority | Processing Pipeline | Backends | Plugin |
+-----+-----+-----+-----+-----+
| splunk_windows | 20 | Splunk Windows log source conditions | splunk, splunk_spl2 | splunk |
| splunk_sysmon_acceleration | 25 | Splunk Windows Sysmon search acceleration keywords | splunk, splunk_spl2 | splunk |
| splunk_cim | 20 | Splunk CIM Data Model Mapping | splunk, splunk_spl2 | splunk |
+-----+-----+-----+-----+-----+
PS C:\WINDOWS\system32>

```

Figure 3 - Available Splunk processing pipelines.

Sigma-to-Splunk Conversion

After correcting a YAML formatting issue, the rule converted successfully using the Splunk Windows pipeline.

```
& "C:\Users\Dulan\AppData\Local\Python\pythoncore-3.14-64\Scripts\sigma.exe" convert -t splunk -p splunk_windows C:\SOC-Lab\lab13_encoded_powershell.yaml
```

Generated SPL:

ScriptBlockText IN ("*FromBase64String*", "*EncodedCommand*", "*ToBase64String*")

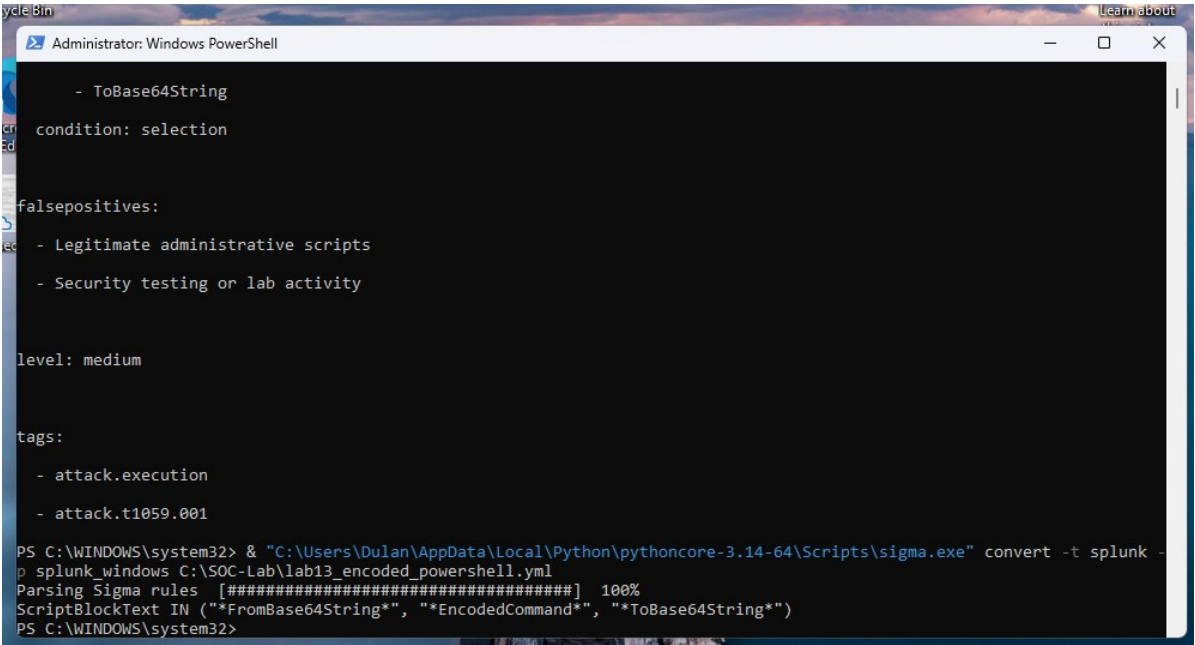


Figure 4 - Successful Sigma conversion and generated Splunk SPL.

PowerShell Telemetry Ingestion

The initial Splunk search returned no PowerShell events because the Universal Forwarder was only collecting the Security channel. The PowerShell Operational channel was then added to inputs.conf:

```
[WinEventLog://Microsoft-Windows-PowerShell/Operational]
disabled = 0
index = main
renderXml = true
```

After restarting the Splunk Universal Forwarder, Event ID 4104 telemetry began arriving in Splunk.

Detection Validation in Splunk

A fresh controlled PowerShell test generated Base64-related script-block activity. Searching Splunk for the detection term **ToBase64String** returned matching Event ID 4104 events.

Validation item	Result
Windows Event ID	4104 - PowerShell Script Block Logging
Splunk source	WinEventLog:Microsoft-Windows-PowerShell/Operational
Splunk sourcetype	XmlWinEventLog:Microsoft-Windows-PowerShell/Operational
Matched indicator	ToBase64String
Detection outcome	Successful match in controlled lab telemetry

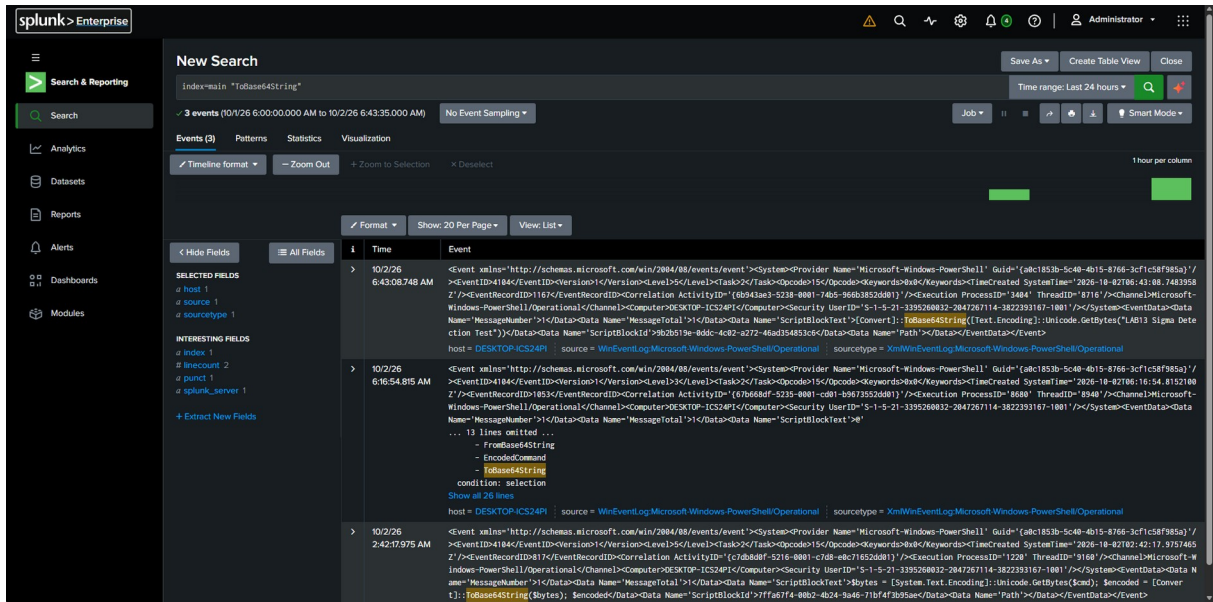


Figure 5 - Splunk matching PowerShell Event ID 4104 telemetry containing ToBase64String.

SOC Analyst Assessment

Item	Assessment
Detection severity	Medium
Test verdict	Detection logic validated
Observed activity	Authorized lab activity
Escalation for this test	No
Production use	Requires contextual triage because encoding can be legitimate or malicious

Encoded or Base64-related PowerShell is not automatically malicious. The rule is intended to surface potentially suspicious script-block activity for analyst review. Legitimate administrators, deployment tooling, security products, and test scripts may generate similar patterns.

Detection Engineering Lessons

- Sigma provides a vendor-neutral way to describe detection logic and can be converted into platform-specific queries.
- A valid rule still needs the correct log source and telemetry pipeline to work in a SIEM.
- Detection validation should include both rule conversion and a controlled event that confirms matching telemetry.
- False positives must be documented because suspicious techniques such as PowerShell encoding also have legitimate uses.
- MITRE ATT&CK tags describe observed behavior and should not be treated as proof of malicious intent.

Final Conclusion

Lab 13 successfully demonstrated an end-to-end detection engineering workflow: Sigma rule creation, YAML correction, Sigma CLI setup, Splunk backend installation, processing-pipeline selection, Sigma-to-SPL conversion, PowerShell log ingestion, and validation against Event ID 4104 telemetry. The final rule detects encoded/Base64-related PowerShell indicators and maps the behavior to MITRE ATT&CK T1059.001. The controlled test matched successfully in Splunk.

Safety note: All validation activity was performed in an authorized lab environment using harmless PowerShell commands. No malicious payloads were executed.